

Lab exercises

Part 05 —  Generative AI · 05-genai/lab/EXERCISES.md

Lab — Generative AI

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. Everything here builds on `../code/diffusion_from_scratch.py` and `../code/guidance_and_sampling.py` — read both once before starting.

1. Warm-up: read the numbers you were given (45 min)

Run both scripts. Then, without changing any code:

- **1a.** In section 2 of `diffusion_from_scratch.py`, find the timestep `t` at which `sqrt(abar_t)` first drops below `0.1`. What fraction of `T` is that? Why does the schedule spend so many steps near the low-noise end instead of linearly destroying signal?
- **1b.** The training printout floors near $\text{MSE} \approx 0.27\text{--}0.3$, not 0.0 . Explain in one sentence why a denoiser that has converged perfectly still cannot reach $\text{MSE} = 0$.
- **1c.** In `guidance_and_sampling.py`'s Q1 table, DDIM at 20 steps and DDPM at 200 steps should have precision/recall within roughly the same ballpark. State the actual numbers you got and the speedup factor in network evaluations.

Deliverable: three short answers (one per part), with your actual numbers quoted, not the ones above — reruns with a different NumPy version or seed will differ slightly.

2. Change the noise schedule (1.5h)

`DiffusionSchedule` uses a linear `beta` schedule. Add a **cosine schedule** (Nichol & Dhariwal, 2021 — see `../papers/PAPERS.md`):

```
abar_t = cos( ((t/T + s) / (1+s)) * pi/2 ) ** 2,    s ~ 0.008
```

- Plot (or print, in a table) `abar_t` vs. `t` for both schedules.
- Retrain the unconditional denoiser under the cosine schedule with the same `iters/lr`.
- Compare final precision/recall proxies against the linear-schedule baseline from script 1.

Checks: - The cosine schedule should destroy signal *more slowly* near `t=0` and `t=T` and faster in the middle — say in one sentence why that matters more for high-resolution images than for this 2D toy problem. - Report whether the cosine schedule improved, hurt, or didn't meaningfully change the precision/recall proxy here, and one hypothesis for why the effect is small on a 2D dataset.

3. Guidance scale, past where the script stops (1h)

`guidance_and_sampling.py`'s Q2 sweep goes up to $w=32$. Extend it to $w = 64, 128, 256$.

Deliverable: an extended table (your sweep, printed) plus two sentences: - What happens to the `diversity` (mean pair dist) column at extreme w , and does it match the "squeezes toward a narrower region" story in the script's closing printout, or contradict it? Quote your numbers. - If it contradicts the story, propose one concrete explanation tied to the guidance equation $\text{eps_guided} = \text{eps_u} + w * (\text{eps_c} - \text{eps_u})$ and what happens numerically to x when w is large and $(\text{eps_c} - \text{eps_u})$ is not exactly zero-mean noise.

This exercise has no "expected" answer key — the point is to notice when a printed narrative and printed numbers disagree, and to debug which one is right. That skill transfers directly to reading papers.

4. Add a third class (2h)

`diffusion_from_scratch.py`'s conditional model uses 2 classes (the two moons). Generate a third cluster (a Gaussian blob well separated from both moons), retrain the conditional denoiser with a 3-dimensional one-hot condition and `p_uncond=0.15`, and sample all three classes under guidance.

Checks: - Report per-class fidelity (as the script does with `classify_by_centroid`, extended to 3 classes) at $w=4$. - Pick one class and sweep w from 0 to 16. At what w does fidelity cross 90%? Is it the same threshold as the two-class case in the original script? If not, hypothesize why (hint: think about how "distance to the nearest wrong class" changes with three classes vs. two).

5. Implement DDIM with $\eta > 0$ (2h)

The provided `ddim_sample` in `guidance_and_sampling.py` hardcodes $\eta=0$ (fully deterministic). Extend it to accept an η parameter that interpolates between DDIM ($\eta=0$) and full DDPM-like stochasticity ($\eta=1$) by injecting scaled noise at each step (see the DDIM paper, section 4, for the variance term).

Checks: - At $\eta=0$, your function's output should match the original bit-for-bit given the same seed. - At $\eta=1$ with the full step count, precision/recall should land close to plain DDPM's. - Sweep η at a fixed step count (e.g. 50) and report whether determinism ($\eta=0$) or stochasticity ($\eta=1$) gives the better precision/recall proxy at that step count. State which you'd ship in a product that promises "same seed, same image."

6. Precision/recall proxy vs. FID and CLIP score (1h, no code — reading + writing)

`nn_distance/precision_recall_proxy` in `diffusion_from_scratch.py` is a cheap 2D stand-in for FID.

- **6a.** Explain in your own words why nearest-neighbor distance in raw 2D coordinates is a reasonable quality proxy here but would be a *terrible* idea for comparing two 512x512 images pixel-by-pixel. What does FID's Inception-feature step buy you that raw pixel distance doesn't?

- **6b.** CLIP score measures image-prompt cosine similarity, not image-image similarity. Design (on paper, no code required) a toy analogue of CLIP score for this 2D moons dataset: what would play the role of "prompt embedding," and what failure mode from notes/02 (§5) would your toy metric reproduce?

Deliverable: two short paragraphs.

7. Capstone — a text-conditioned toy pipeline (3h)

Combine everything above into one script:

1. Three or more labeled 2D clusters, each tagged with a short text label (e.g. "upper moon", "lower moon", "center blob").
2. A tiny "text encoder" — even a fixed one-hot-per-label lookup is fine, the point is the pipeline shape, not real NLP
3. A conditional DDPM denoiser trained with CFG-style condition dropout.
4. A `generate(prompt: str, guidance_scale: float, sampler: "ddpm"|"ddim", steps: int)` function that routes the label through the "text encoder," conditions the denoiser, and samples with your choice of sampler.
5. A small report (printed, not necessarily a file): for 3 prompts x 2 guidance scales x 2 samplers, fidelity, diversity, and wall time.

Check: hand someone else your `generate()` function signature alone. If they can predict what changing each argument will do to the output (without reading your implementation), you've built something that actually mirrors how a real `diffusers` pipeline call is structured — see `../resources/RESOURCES.md` for the real library once you're ready to compare.